Introduction

Circular List

Data Structures and its Applications (UE25CS52A)

N S Kumar
Professor
Department of Computer Science & Engineering (AI & ML)

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Circular List

1. What is a common way to detect the end of traversal in a circular linked list?

a) Check current == NULL

b) Check current->next_ == NULL

c) Check whether current has returned to the starting node

d) Check whether current->key_ == 0

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Circular List

1. What is a common way to detect the end of traversal in a circular linked list?

a) Check current == NULL

b) Check current->next_ == NULL

c) Check whether current has returned to the starting node

d) Check whether current->key_ == 0

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Circular List

2. while (temp != NULL) {
```
        // some other code
        temp = temp->next_;
```
}

temp is a pointer to one of the nodes. Comment.

a) Will become infinite loop

b) Will traverse all the nodes and exit the loop

c) Will traverse all but one node and exit the loop

d) Will result in undefined behaviour

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Circular List

2. while (temp != NULL) {
    // some other code
    temp = temp->next_;
}

temp is a pointer to one of the nodes. Comment.

a) Will become infinite loop

b) Will traverse all the nodes and exit the loop

c) Will traverse all but one node and exit the loop

d) Will result in undefined behaviour

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Circular List

3. node_t* temp = ptr_clist->head_; node_t* temp2 = ptr_clist->head_;

```
do {
        temp = temp->next_;
        temp2 = temp2->next_->next_;
} while (temp2 != ptr_clist->head_);
```

Comment.

a) temp2 will always point to a node half way across from the head

b) Both temp and temp2 may be at the head when the loop terminates

c) This may result in an infinite loop if the number of nodes is even

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Circular List

3. node_t* temp = ptr_clist->head_; node_t* temp2 = ptr_clist->head_;

```
while (temp2 != prt_clist->head_) {
        temp = temp->next_;
        temp2 = temp2->next_->next_;
}
```

Comment.

a) temp2 will always point to a node half way across from the head

b) Both temp and temp2 may be at the head when the loop terminates

c) This may result in an infinite loop if the number of nodes is even

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Circular List

4. Consider a circular list containing only one node.
Which of the following is true?

a) ptr_list->head_ == NULL

b) ptr_list->head_->next_ == NULL

c) ptr_list->head_ == ptr_list->head_->next_

d) ptr_list->head_ may point to any node

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Circular List

4. Consider a circular list containing only one node.
Which of the following is true?

a) ptr_list->head_ == NULL

b) ptr_list->head_->next_ == NULL

c) ptr_list->head_ == ptr_list->head_->next_

d) ptr_list->head_ may point to any node

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Circular List

5. In a circular linked list with head_ referring a node in the list, what happens when the only node is deleted?

a) head_ remains unchanged

b) head_ becomes NULL

c) head_ ->next_ becomes NULL

d) A new node must be created

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Circular List

5. In a circular linked list with head_ referring a node in the list, what happens when the only node is deleted?

a) head_ remains unchanged

b) head_ becomes NULL

c) head_ ->next_ becomes NULL

d) A new node must be created

© Confidential Information of PES University. All Rights Reserved - 2026

Circular List : concept

- This is a list
- The last node points to the first
    - Have a pointer to one of the nodes
    - first and last are not very meaningful
- When the elements form a cycle or need to be processed repeatedly,

  the last element should naturally lead back to the first.

© Confidential Information of PES University. All Rights Reserved - 2026

Circular List: interface

- Initialize / Deinitialize the list
- Insert anywhere in the list
- Delete anywhere in the list
- Traverse forward
- Search for a designated key

© Confidential Information of PES University. All Rights Reserved - 2026

Circular List: structure

```
//Same as that of a singly linked list
struct node
{
        int key_;
        struct node *next_;
};
typedef struct node node_t;
struct clist
{
        node_t *head_;
};
typedef struct clist clist_t;
```


© Confidential Information of PES University. All Rights Reserved - 2026

Circular List: implementation

- Initialize / Deinitialize the list
  - Initialize : Make head_ NULL
  - Deinitialize : Traverse and remove nodes until back to the first node; Remove that node; ground the head_.
- Insert after/before the first node
  - Insertion is exactly the same
  - Modify the head_ suitably
- Delete after the first node
- Traverse
  - Prefer a do…while loop
  - Stop when the first node is reached again
- Search for a designated key

© Confidential Information of PES University. All Rights Reserved - 2026

Circular List: Applications

- Multiplayer Turn Management
- Music Playlist – repeat continuously
- Image Slideshow - repeat continuously
- Token Passing - ring-based algorithms

© Confidential Information of PES University. All Rights Reserved - 2026

Circular List: Josephus problem : problem statement

Josephus Problem:

1. Find the last remaining survivor from n people standing in a circle.
2. Counting begins at a specific person
3. Moves around the circle in a fixed direction
4. In each step, k-1 people are skipped, and the k-th person is removed.
5. The process repeats until only one person is left.
6. Find that person.

© Confidential Information of PES University. All Rights Reserved - 2026

Circular List: Josephus problem : algorithm

The steps involved in solving this problem are:

1. n people are numbered from 1 to n in order and are placed in a circle
2. Starting from a designated start point, we count to kth person.
3. The kth person is eliminated
4. Counting and elimination steps are repeated until a single person is left
5. Original number of that person is the solution.

© Confidential Information of PES University. All Rights Reserved - 2026

Circular List: Josephus problem : Example

- **n = 7** people
- **k = 3** → eliminate every 3rd person

Step	Counting	Eliminated
1	1 → 2 → 3	3
2	4 → 5 → 6	6
3	7 → 1 → 2	2
4	4 → 5 → 7	7
5	1 → 4 → 5	5
6	1 → 4 → 1	1

4 is the survivor

© Confidential Information of PES University. All Rights Reserved - 2026

THANK YOU

N S Kumar
Professor

Department of Computer Science & Engineering (AI & ML)

Introduction

Doubly Linked List
Data Structures and its Applications (UE25CS52A)

N S Kumar
Professor
Department of Computer Science & Engineering (AI & ML)

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Doubly Linked List

1. In a non-circular doubly linked list, the prev pointer of the first node contains:

a) Pointer to the last node

b) Pointer to the second node

c) NULL

d) Pointer to itself

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Doubly Linked List

1. In a non-circular doubly linked list, the prev pointer of the first node contains:

a) Pointer to the last node

b) Pointer to the second node

c) NULL

d) Pointer to itself

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Doubly Linked List

2. Suppose p is a pointer to a node in the middle of the list and we want to insert new node pointed to by temp immediately after p. Which sequence is right with respect to p?

a) p->next_ = temp->next_; p->prev_ = temp->prev_;

b) temp->next_ = p->next_; temp->prev_ = p->prev_;

c) temp->next_ = p->next_; temp->prev_ = p;

d) temp->next_ = p; temp->prev_ = p->prev_;

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Doubly Linked List

2. Suppose p is a pointer to a node in the middle of the list and we want to insert new node pointed to by temp immediately after p. Which sequence is right with respect to p?

a) p->next_ = temp->next_; p->prev_ = temp->prev_;

b) temp->next_ = p->next_; temp->prev_ = p->prev_;

c) temp->next_ = p->next_; temp->prev_ = p;

d) temp->next_ = p; temp->prev_ = p->prev_;

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Doubly Linked List

3. When a new node is added to an empty list,

a) neither head_ not tail_ of the DLL will change

b) both head_ and tail_ of the DLL could change

c) only head_ of the DLL would change

d) Only tail_ of the DLL would change

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Doubly Linked List

3. When a new node is added to an empty list,

a) neither head_ not tail_ of the DLL will change

b) both head_ and tail_ of the DLL could change

c) only head_ of the DLL would change

d) Only tail_ of the DLL would change

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Doubly Linked List

4. temp->next_ = p->next_; temp->prev_ = p; p->next_ = temp;

While inserting a new node temp not at the end of the list, which statement is missing?

a) p->next_->prev_ = temp;

b) p->prev_ = temp;

c) p->next_ = NULL;

d) p->next_ = NULL;

© 2026

Quiz : Doubly Linked List

4. temp->next = p->next; temp->prev = p; p->next = temp;

While inserting a new node temp not at the end of the list, which statement is missing?

a) p->next_->prev_ = temp;

b) p->prev_ = temp;

c) p->next_ = NULL;

d) p->next_ = NULL;

© 2026

Quiz : Doubly Linked List

5. p->prev->next = p->next;p->next->prev = p->prev;

a) Deletes the node to which p points to and handles memory correctly

b) Deletes the node to which p points to, but results in memory leak

c) Breaks the list and part of the list is lost

d) Deletes the node in the beginning, or at the end or the middle

© 2026

Quiz : Doubly Linked List

5. p->prev->next = p->next;p->next->prev = p->prev;

a) Deletes the node to which p points to and handles memory correctly

b) Deletes the node to which p points to, but results in memory leak

c) Breaks the list and part of the list is lost

d) Deletes the node in the beginning, or at the end or the middle

© 2026

Doubly Linked List : a few points

- Each node allows movement to the previous previous and the nodes
  - Normally called prev_ and next_ standing for previous and next
- The list node has two pointers – one to the first node and one to the last node
  - Normally called head_ and tail_
- Allows for traversal in both the directions forward and backward

© Confidential Information of PES University. All Rights Reserved - 2026

Doubly Linked List : interface

- Initialize / Deinitialize the list
- Insert anywhere in the list
- Delete anywhere in the list
- Traverse forward or backward
- Search for a designated key

© Confidential Information of PES University. All Rights Reserved - 2026

Doubly Linked List : key structure

```
struct node {
   int key_;
   struct node *prev_;
   struct node *next_;
};
typedef struct node node_t;
struct DLL {
            node_t *head_;
            node_t *tail_;
};
typedef struct DLL dll_t;
```


© Confidential Information of PES University. All Rights Reserved - 2026

Doubly Linked List : implementation

- Initialize / Deinitialize the list
  - Initialize : Ground both head_ and tail_
  - Deinitialize : traverse and free the nodes. Reinitialize the list

- Insert anywhere in the list
  - Take care of the following cases
    - Empty list : both head_ and tail_ should change
    - Add in the beginning : head_ should change
    - Add at the end : tail_ should change
    - Add in the middle
  - Observe there will be 4 changes to the pointers

© Confidential Information of PES University. All Rights Reserved - 2026

Doubly Linked List : implementation

- Delete anywhere in the list
  - In the beginning : head_ should change
  - At the end : tail_ should change
  - Anywhere in the middle
  - Deleting the only node : both head_ and tail_ should change
- Traverse forward or backward
- Search for a designated key

© Confidential Information of PES University. All Rights Reserved - 2026

Singly vs. Doubly Linked Lists

Worst-Case Operation (n nodes)	Singly Linked List	Doubly Linked List
Move Forward	1 step	1 step
Move Backward	n steps (Requires re-traversal)	1 step (Instant fallback)
Insert Before a Node	n steps (Find predecessor)	1 step (Using prev_ link)
Delete a Designated Node	n steps (Find predecessor)	1 step (Local isolation)
Searching for a Key	n steps	n steps

© Confidential Information of PES University. All Rights Reserved - 2026

Doubly Linked List : Advantages

1. Bidirectional traversal
2. Easy deletion
3. Easy insertion before or after a known node
4. Natural representation of navigation

© Confidential Information of PES University. All Rights Reserved - 2026

Doubly Linked List : Disadvantages

1. Extra memory
2. More pointer manipulation
3. More opportunities for bugs
4. Still has linear search

© Confidential Information of PES University. All Rights Reserved - 2026

Doubly Linked List : Applications

1. Browser History
2. Music Player Playlist
3. Undo and Redo
4. LRU cache

© Confidential Information of PES University. All Rights Reserved - 2026

THANK YOU

N S Kumar
Professor

Department of Computer Science & Engineering (AI & ML)

Introduction

Queue using a linked list
Data Structures and its Applications (UE25CS52A)

N S Kumar
Professor
Department of Computer Science & Engineering (AI & ML)

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Queue

1. On dequeue

a) Only head of the queue is changed

b) Only tail of the queue is changed

c) Both head and tail could change

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Queue

1. On dequeue

a) Only head of the queue is changed

b) Only tail of the queue is changed

c) Both head and tail could change

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Queue

2. To make both enqueue and dequeue efficient,

a) The pointer of each node will point to the node which joined the queue later

b) The pointer of each node will point to the node which joined the queue earlier

c) The pointer can point to either node before or node after

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Queue

2. To make both enqueue and dequeue efficient,

a) The pointer of each node will point to the node which joined the queue later

b) The pointer of each node will point to the node which joined the queue earlier

c) The pointer can point to either node before or node after

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Queue

3. Enqueue will change,

a) Only the head

b) Only the tail

c) Could change both the head and the tail

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Queue

3. Enqueue will change,

a) Only the head

b) Only the tail

c) Could change both the head and the tail

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Queue

4. If both the front and the rear are equal,

a) Implies that definitely the queue is empty

b) Implies that definitely the queue has one node

c) Implies either empty queue or one node queue

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Queue

4. If both the front and the rear are equal,

a) Implies that definitely the queue is empty

b) Implies that definitely the queue has one node

c) Implies either empty queue or one node queue

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Queue

5. Calling free on the front of a non empty queue,

a) Removes the first node and the rest of the queue is accessible

b) Remove the whole queue

c) Front automatically will point to the next node in the queue

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Queue

5. Calling free on the front of a non empty queue,

a) Removes the first node and the rest of the queue is accessible

b) Remove the whole queue

c) Front automatically will point to the next node in the queue

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Queue: interface

- Initialize / Deinitialize the queue
- enqueue
- dequeue
- Is_empty
- Is_full

© Confidential Information of PES University. All Rights Reserved - 2026

queue: structure

```
struct node
{
        int key_;
        struct node* next_;
};
typedef struct node node_t;
struct queue
{
        node_t* front_;
        node_t* rear_;
};
typedef struct queue queue_t;
```


© Confidential Information of PES University. All Rights Reserved - 2026

Queue: implementation

- Initialize / Deinitialize the queue
  - Initialize : make front and rear NULL
  - Deinitialize : deque all the nodes. make head and tail NULL
- Enqueue
  - Add at the rear
  - Modify rear
  - First node => modify front
- Deque
  - Remove the front
  - Modify front
  - Only node => modify rear

© Confidential Information of PES University. All Rights Reserved - 2026

Queue: Applications

- CPU scheduling : processes in batch processing usually wait in a queue
- Printer Spooler : collects the jobs for printing in a queue
- Breadth First search : Looking for a key in a tree structure
- Simulation models : queues to simulate random events in queueing theory
- Message queues : sharing information in operating systems
- Toll booth : waiting of vehicles to pass through a toll booth

© Confidential Information of PES University. All Rights Reserved - 2026

THANK YOU

N S Kumar
Professor

Department of Computer Science & Engineering (AI & ML)

Introduction

Queue using a circular list using an array
Data Structures and its Applications (UE25CS52A)

N S Kumar
Professor
Department of Computer Science & Engineering (AI & ML)

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Queue

1. In our implementation, on initialization

a) Front and rear should be -1

b) Front and rear should be 0

c) Front and rear could be any valid index

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Queue

1. In our implementation, on initialization

a) Front and rear should be -1

b) Front and rear should be 0

c) Front and rear could be any valid index

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Queue

2. On deque, queue is empty if

a) Front and rear are equal after incrementing front

b) Front and rear are equal after incrementing rear

c) Front and rear are equal before incrementing rear

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Queue

2. On deque, queue is empty if

a) Front and rear are equal after incrementing front

b) Front and rear are equal after incrementing rear

c) Front and rear are equal before incrementing rear

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Queue

3. deque need not consider this as a special case

a) Empty queue

b) Queue with one node

c) Queue with number of nodes

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Queue

3. deque need not consider this as a special case

a) Empty queue

b) Queue with one node

c) Queue with number of nodes

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Queue

4. To support different type of component in the queue,

a) The type of front should change

b) The type of rear should change

c) The type of key should change

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Queue

4. To support different type of component in the queue,

a) The type of front should change

b) The type of rear should change

c) The type of key should change

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Queue

5. Deinitialize the queue

a) Requires number of operations as the number of components in the queue

b) Requires a few operations; does not depend on the number of components in the queue

c) Requires a looping structure

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Queue

5. Deinitialize the queue

a) Requires number of operations as the number of components in the queue

b) Requires a few operations; does not depend on the number of components in the queue

c) Requires a looping structure

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

queue: structure

```
#define MAXSIZE 5
struct queue
{
        int key_[MAXSIZE];
        int front_;
        int rear_;
};
typedef struct queue queue_t;
```


© Confidential Information of PES University. All Rights Reserved - 2026

Queue: implementation

- Initialize / Deinitialize the queue
  - Initialize : make head and tail 0
  - Deinitialize : make head and tail 0
- Enqueue
  - Add at the rear
  - Increment and check whether queue is full
  - First node => modify front
- Deque
  - Check whether the queue is empty
  - If not increment front
  - Remove the front
  - Modify front

© Confidential Information of PES University. All Rights Reserved - 2026

Queue: implementation

- Initialize / Deinitialize the queue
  - Initialize : make head and tail 0
  - Deinitialize : make head and tail 0

```
void init_queue(queue_t *ptr_queue)
{
        // could be any valid index in the array
        ptr_queue->rear_ = ptr_queue->front_ = 0;
}
void deinit_queue(queue_t *ptr_queue) // NO LOOP!
{
        ptr_queue->rear_ = ptr_queue->front_ = 0;
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Queue: implementation

- is_full and is_empty : same code!!

```
// to be called before deque before incrementing
int is_empty(queue_t *ptr_queue)
{
        return ptr_queue->front_ == ptr_queue->rear_;
}

// to be called in enque after incrementing
int is_full(queue_t *ptr_queue)
{
        return ptr_queue->front_ == ptr_queue->rear_;
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Queue: implementation : **enque and deque**

```
void enque(queue_t *ptr_queue, int key)
{
        int temp;
        temp = ++ptr_queue->rear_ % MAXSIZE;
        ptr_queue->rear_ = temp;
        if(! is_full(ptr_queue))  {
                ptr_queue->key_[temp] = key;
        }
        else {
                ptr_queue->rear_ = (ptr_queue->rear_ - 1) % MAXSIZE;
                printf("queue full; cannot push\n");
        }
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Queue: implementation : enque and deque

```
int deque(queue_t *ptr_queue)
{
        if(! is_empty(ptr_queue)) {
                int key;  int temp = ++ptr_queue->front_ % MAXSIZE;
                key = ptr_queue->key_[temp];
                ptr_queue->front_ = temp;
                return key;
        } else {
                printf("queue empty; cannot pop\n");
                return '\0';
        }
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Queue: Applications

- CPU scheduling : processes in batch processing usually wait in a queue
- Printer Spooler : collects the jobs for printing in a queue
- Breadth First search : Looking for a key in a tree structure
- Simulation models : queues to simulate random events in queueing theory
- Message queues : sharing information in operating systems
- Toll booth : waiting of vehicles to pass through a toll booth

© Confidential Information of PES University. All Rights Reserved - 2026

THANK YOU

N S Kumar
Professor

Department of Computer Science & Engineering (AI & ML)

Introduction

Priority Queue
Data Structures and its Applications (UE25CS52A)

N S Kumar
Professor
Department of Computer Science & Engineering (AI & ML)

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Priority Queue

1. **. Which statement best describes a priority queue ADT?**

a) Elements are always processed in FIFO order
b) Elements are processed according to their associated priorities
c) Elements are processed in LIFO order
d) Elements are stored only in sorted order

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Priority Queue

1. **. Which statement best describes a priority queue ADT?**

a) Elements are always processed in FIFO order

b) Elements are processed according to their associated priorities

c) Elements are processed in LIFO order

d) Elements are stored only in sorted order

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Priority Queue

2. In a max-priority queue, which element is removed first

a) One with the highest priority which came later
b) One with the highest priority which came earlier
c) Any one with the highest priority
d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Priority Queue

2. In a max-priority queue, which element is removed first

a) One with the highest priority which came later

b) One with the highest priority which came earlier

c) Any one with the highest priority

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Priority Queue

3. **Which statement correctly distinguishes a priority queue from an ordinary queue?**

a) A priority queue allows insertion but not deletion

b) An ordinary queue always uses priorities

c) A priority queue determines deletion primarily by priority

d) An ordinary queue cannot be implemented using an array

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Priority Queue

3. **Which statement correctly distinguishes a priority queue from an ordinary queue?**

a) A priority queue allows insertion but not deletion

b) An ordinary queue always uses priorities

c) A priority queue determines deletion primarily by priority

d) An ordinary queue cannot be implemented using an array

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Priority Queue

4. Which statement about equal priorities is correct?

a) Equal-priority elements are not allowed
b) All should be selected together
c) The queue must always use FIFO for equal priorities
d) The queue must always use LIFO for equal priorities

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Priority Queue

4. Which statement about equal priorities is correct?

a) Equal-priority elements are not allowed
b) All should be selected together
c) The queue must always use FIFO for equal priorities
d) The queue must always use LIFO for equal priorities

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Priority Queue

5. Which algorithm may not use priority queue?

a) Patient management
b) Scheduling processes in an operaring system
c) Managing function calls
d) Selecting the next print job from the spool

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Priority Queue

5. Which algorithm may not use priority queue?

a) Patient management
b) Scheduling processes in an operaring system
c) Managing function calls
d) Selecting the next print job from the spool

© Confidential Information of PES University. All Rights Reserved - 2026

Priority Queue: concept

- A priority queue is an abstract data type in which each element is associated with a priority.
- A priority queue removes the component with the highest priority first.
- A priority is a number associated with each element
- Higher the numeric value, could mean higher the priority.
- Or lower the numeric value, could mean higher priority

© Confidential Information of PES University. All Rights Reserved - 2026

Priority Queue: some examples

- Patient treatment
- Scheduling of processes

© Confidential Information of PES University. All Rights Reserved - 2026

Priority Queue: interface

initialize() deinitialize()	Create an empty queue Make the queue empty
enqueue(x)	Insert x We may add in number of ways. This in turn would affect dequeue
dequeue()	Remove the element with the highest priority. This in turn depends how enqueue operation was done
isEmpty()	Check whether the queue is empty
isFull()	Check whether the queue is full

© Confidential Information of PES University. All Rights Reserved - 2026

Priority Queue: Possible Implementations

- Unodered list
- Ordered list : ordered based on priority
- Unordered array
- Ordered array : ordered based on priority
- Heap tree

© Confidential Information of PES University. All Rights Reserved - 2026

Priority Queue: Applications of priority queue :

- CPU Scheduling of processes
- Hospital Emergency Services
- Printer Scheduling
- Network Packet Scheduling
- Dijkstra's Shortest Path Algorithm
- Prim's Minimum Spanning Tree Algorithm
- Event-Driven Simulation
- Huffman Coding

© Confidential Information of PES University. All Rights Reserved - 2026

THANK YOU

N S Kumar
Professor

Department of Computer Science & Engineering (AI & ML)

Priority Queue : unordered list implementation

A priority queue is an abstract data type in which each element is associated with a priority.

Unlike an ordinary queue, where elements are removed according to FIFO (First-In, First-Out) order, a priority queue removes the component with the highest priority first.

A priority is a number associated with each component. Higher the numeric value, could mean higher the priority. Or lower the numeric value could mean higher priority.

In our examples, we will assume that higher the numeric value, higher the priority.

Let us relook at the interface of the priority queue before discussing the implementation using unordered list.

Interface of the priority queue:

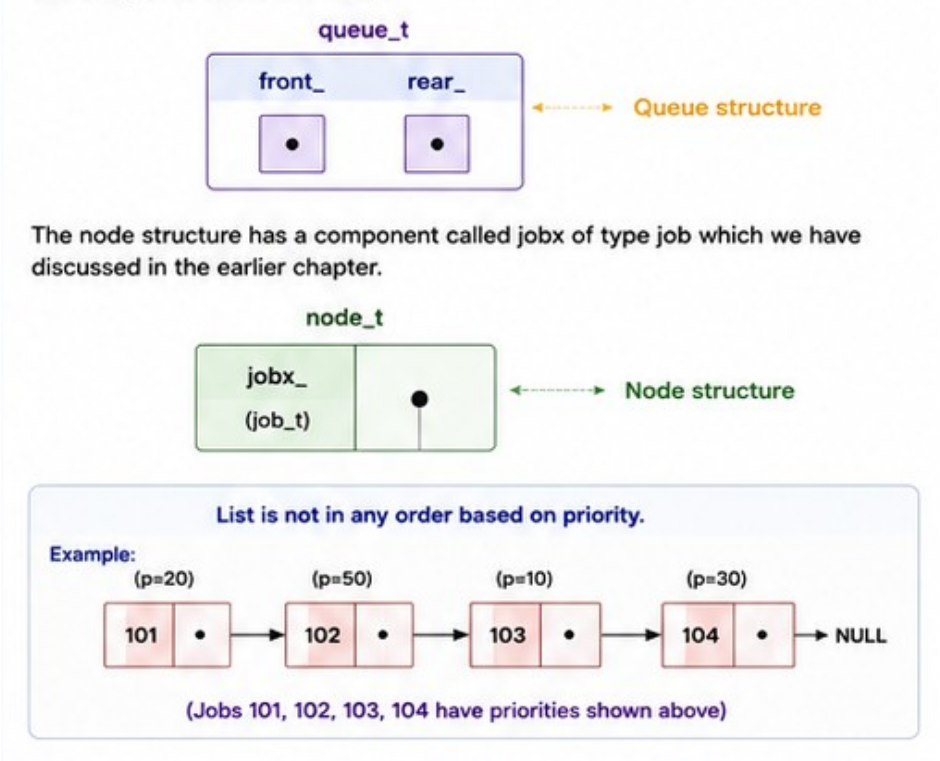
The interface of a priority queue is same as that of queue. Component x would have a priority associated with it.

initialize()	Create an empty queue
deinitialize()	Make the queue empty
enqueue(x)	Insert x We may add in number of ways. This in turn would affect dequeue
dequeue()	Remove the element with the highest priority. This in turn depends how enqueue operation was done
isEmpty()	Check whether the queue is empty
isFull()	Check whether the queue is full

Implementation of priority queue:

The structure of the priority queue is same as that of queue using a linked list. It has two pointers to the node structure – one points to the beginning - called front and one points to the end – called the rear.

```
struct queue
{
        node_t* front_;
        node_t* rear_;
};
typedef struct queue queue_t;
```



The node structure has a component called jobx of type job which we have discussed in the earlier chapter.

```
struct node
{
        job_t jobx_;
        struct node* next_;
};
typedef struct node node_t;
```

The interface of priority queue remains same as that of queue. The implementation would dépend on the underlying data structure to support enque and deque appropriately.

```
void init_queue(queue_t *ptr_queue);
void deinit_queue(queue_t *ptr_queue);
void enque(queue_t *ptr_queue, job_t *ptr_job);
job_t deque(queue_t *ptr_queue);
int is_empty(queue_t *ptr_queue);
int is_full(queue_t *ptr_queue);
```

The list is not in any order based on priority. Let us discuss each one of these operations.

Initialize:

Both the front and the rear of the queue are grounded.

```
 void init_queue(queue_t *ptr_queue)
{
        ptr_queue->rear_ = ptr_queue->front_ = NULL;
}
```

Deinitialize :

This is one of the ways of removing all the elements in the queue. Deque each element until the queue becomes empty. As deque is a costly operation, we could do it better. Can you try it out?

Look at this code below. Do you think this is required?

```
ptr_queue->rear_ = ptr_queue->front_ = NULL;
```

```
void deinit_queue(queue_t *ptr_queue)
{
        while(! is_empty(ptr_queue))
        {
                deque(ptr_queue);
        }
        ptr_queue->rear_ = ptr_queue->front_ = NULL;
}
```

Is full :

This is a trick. Try allocating. If it succeeds, then there is memory available. Release the memory allocated just for the testing purpose.

```
int is_full(queue_t *ptr_queue)
{
   node_t* temp;
  temp = (node_t*)malloc(sizeof(node_t));
   if (temp == NULL) {
      return 1;
   }
   free(temp);
   return 0;
}
```

Is empty:

Check whether the front of the queue is NULL.

Can we check the rear of the queue in stead?

Would front and rear being equal indicate that the queue is empty?

```
int is_empty(queue_t *ptr_queue)

{

        return ptr_queue->front_ == NULL;

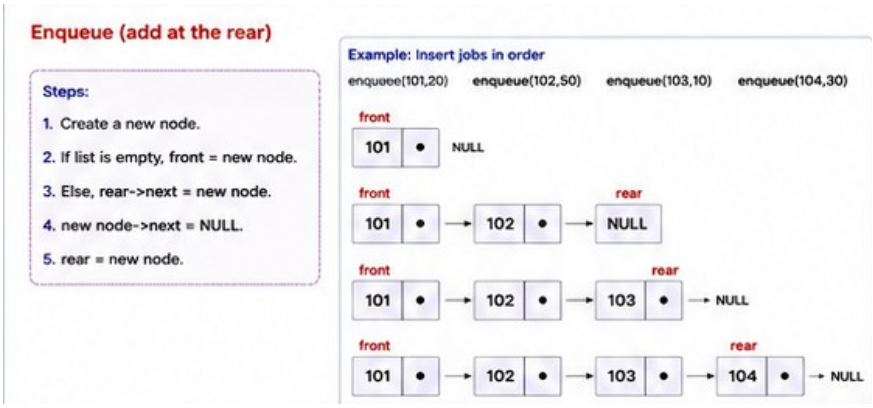
}
```

Enque :

This operation does not consider the priority as the list is unordered. It adds the element at the end.

If queue is full, the program is exited.

Make a node. Populate the node. Add at the rear. Change front if the queue is empty. Otherwise make next of rear the new node added. Ground the next of the new node. Change the rear to point to the new node.



```
void enque(queue_t *ptr_queue, job_t *ptr_job)
{

	if(! is_full(ptr_queue))
	{
		node_t* temp = create_node(ptr_job);
		temp->next_ = NULL;
		if(ptr_queue->rear_ == NULL)
		{
			ptr_queue->front_ = temp;
		}
		else
		{
			ptr_queue->rear_->next_ = temp;
		}
		ptr_queue->rear_ = temp;
	}
	else
	{
		printf("queue full; cannot push\n");
	}
}
```

Deque :

We must find the node with the highest priority. If there are mode nodes with the same highest priority, we should select the job which came first. So, such a node will be closed to the front than the other nodes with the same priority.

If the queue is empty, the program is exited.

There are two parts in the algorithm.

The first is to find the position of the node with the maximum priority. We traverse the normal way.. We travel till the end of the list. We always keep track of the node with the highest priority – with two pointers – max_prev and max_pres.

The second part is to capture the job and then remove the node. If it is the first node, the front has to change. If it is the only node, the rear has to change. Otherwise we should make next of max_prev take the value of next of max_pres. Then release the max_pres node. Then return the job previously captured.

Do convince yourself that the order of operations matters.

A question for you – Can we remove the prev pointer from this algorithm?

```
job_t deque(queue_t *ptr_queue)

{

        if(! is_empty(ptr_queue))

        {

                // traverse

                node_t *prev = NULL;

                node_t *pres = ptr_queue->front_;

                int  max_priority = get_priority(&pres->jobx_);

                node_t *max_prev = NULL;

                node_t *max_pres = pres;

                int priority;

                while(pres != NULL)

                {

                        priority = get_priority(&pres->jobx_);
```

```
        if(priority > max_priority)
        {
                max_priority = priority;
                max_prev = prev;
                max_pres = pres;
        }

        prev = pres;
        pres = pres->next_;
}

job_t jobx;
jobx = max_pres->jobx_;

if(max_prev == NULL)
{
        ptr_queue->front_ = max_pres->next_;
        if(ptr_queue->front_ == NULL)
        {
                ptr_queue->rear_ = NULL;
        }
}
else
{
        max_prev->next_ = max_pres->next_;
        if(max_pres->next_ == NULL) // last node deleted
```

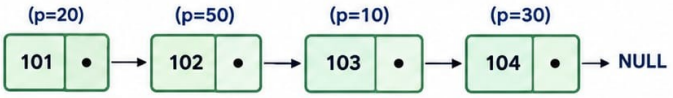
```
                {
                        ptr_queue->rear_ = max_prev;
                }
        }
        free(max_pres);
        return jobx;
    }
    else
    {
        printf("queue empty; cannot pop\n");
        exit(1);
    }
}
```

Dequeue (remove highest priority element)

Part 1: Find the node with highest priority

Keep track of the highest-priority node using two pointers: **max_prev** and **max_pres**.

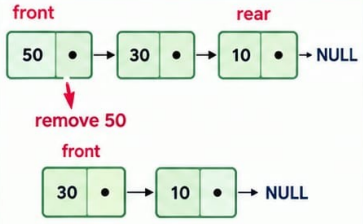
Example list



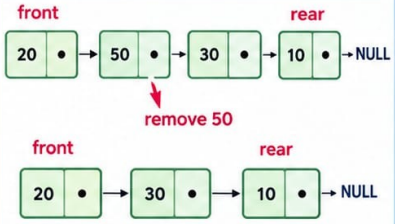
Step	pres (current)	max_prev	max_pres	Explanation
Start	101 (p=20)	NULL	101 (p=20)	Highest so far is 101
1	102 (p=50)	101	102 (p=50)	102 has higher priority
2	103 (p=10)	101	102 (p=50)	No change
3	104 (p=30)	101	102 (p=50)	No change

Part 2: Remove that node

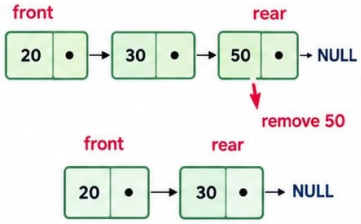
Case A: max_pres is front
(remove first node)



Case B: max_pres is in middle



Case C: max_pres is rear



Efficiency of operations :

Enque : This requires a few operations. We add at the rear without considering the priority.

Deque : This requires walking through the list to find the node with the highest priority. So, the number of operations is proportional to the number of nodes in the queue.

Introduction

Priority Queue : using other implementations Data Structures and its Applications (UE25CS52A)

N S Kumar
Professor
Department of Computer Science & Engineering (AI & ML)

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Priority Queue : using other implementations

1. Which statement best describes enque in a priority queue implemented using an unordered array?

a) Can add in the beginning; would require a few operations independent of the number of elements in the array
b) Can add at the end; would require a few operations independent of the number of elements in the array
c) Should insert in the middle
d) Should depend on the priority

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Priority Queue : using other implementations

1. Which statement best describes enque in a priority queue implemented using an unordered array?

a) Can add in the beginning; would require a few operations independent of the number of elements in the array
b) Can add at the end; would require a few operations independent of the number of elements in the array
c) Should insert in the middle
d) Should depend on the priority

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Priority Queue : using other implementations

2. Which statement best describes enque in a priority queue implemented using an ordered array?

a) Can add in the beginning; would require a few operations independent of the number of elements in the array
b) Can add at the end; would require a few operations independent of the number of elements in the array
c) Should insert in the middle
d) Should depend on the priority

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Priority Queue : using other implementations

2. Which statement best describes enque in a priority queue implemented using an ordered array?

a) Can add in the beginning; would require a few operations independent of the number of elements in the array
b) Can add at the end; would require a few operations independent of the number of elements in the array
c) Should insert in the middle
d) Should depend on the priority

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Priority Queue : using other implementations

3. Which statement best describes enque and deque in a priority queue implemented using an ordered list?

a) Both require operations proportional to the number of elements in the priority queue
b) Both require a few operations independent of the number of elements in the priority queue
c) Enque would require more operations than deque
d) Deque would require more operations than enque

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Priority Queue : using other implementations

3. Which statement best describes enque and deque in a priority queue implemented using an ordered list?

a) Both require operations proportional to the number of elements in the priority queue
b) Both require a few operations independent of the number of elements in the priority queue
c) Enque would require more operations than deque
d) Deque would require more operations than enque

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Priority Queue : using other implementations

4. Deinitializing a priority queue requires

a) A Few operations in an array implementation
b) A few operations in a list implementation
c) A number of operations proportional to the number of elements in an array implementation
d) Freeing of resources in array implementation

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Priority Queue : using other implementations

4. Deinitializing a priority queue requires

a) A Few operations in an array implementation
b) A few operations in a list implementation
c) A number of operations proportional to the number of elements in an array implementation
d) Freeing of resources in array implementation

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Priority Queue : using other implementations

5. Deque in an unordered array implementation of priority queue requires

a) Shifting the elements from right to left
b) Shifting the elements from left to right
c) Just a single assignment – no shifting
d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Priority Queue : using other implementations

5. Deque in our unordered array implementation of priority queue requires

a) Shifting the elements from right to left
b) Shifting the elements from left to right
c) Just a single assignment – no shifting
d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Priority Queue: concept

- A priority queue is an abstract data type in which each element is associated with a priority.
- A priority queue removes the component with the highest priority first.
- A priority is a number associated with each element
- Higher the numeric value, could mean higher the priority.
- Or lower the numeric value, could mean higher priority

© Confidential Information of PES University. All Rights Reserved - 2026

Priority Queue: some examples

- Patient treatment
- Scheduling of processes

© Confidential Information of PES University. All Rights Reserved - 2026

Priority Queue: interface

initialize() deinitialize()	Create an empty queue Make the queue empty
enqueue(x)	Insert x We may add in number of ways. This in turn would affect dequeue
dequeue()	Remove the element with the highest priority. This in turn depends how enqueue operation was done
isEmpty()	Check whether the queue is empty
isFull()	Check whether the queue is full

© Confidential Information of PES University. All Rights Reserved - 2026

Priority Queue: Possible Implementations

- Unodered list
- Ordered list : ordered based on priority
- Unordered array
- Ordered array : ordered based on priority
- Heap tree

© Confidential Information of PES University. All Rights Reserved - 2026

Priority Queue: ordered list

- Implemented using queue implementation
- Could also implement using a list

© Confidential Information of PES University. All Rights Reserved - 2026

Priority Queue: ordered list implementation

- Initialize : make the front and the rear NULL
- Deinitialize : deallocate the nodes and then make the front and the rear NULL
- Is_empty : check front is NULL
- Is_full : check whether memory allocation fails

© Confidential Information of PES University. All Rights Reserved - 2026

Priority Queue: ordered list implementation

- Enque :
  - Make a node
  - Populate
  - Find the position to insert
  - Change front if required
  - Numer of operations depends on the number of elements in the priority queue
- Deque :
  - Copy the job to return
  - Change the front
  - Release the node
  - Suitably adjust front and then rear if required

© Confidential Information of PES University. All Rights Reserved - 2026

Priority Queue: unordered array implementation

- Initialize : make the number of elements in the array 0
- Deinitialize : same as initialize!
- Is_empty : check if the number of elements is 0
- Is_full : check if the number of elements equal to the size of the array

© Confidential Information of PES University. All Rights Reserved - 2026

Priority Queue: unordered array implementation

- Enque :
  - Check if full
  - Add at the end
  - Increase the size
  - Only a few operations
- Deque :
  - Traverse from the beginning
  - Find the leftmost position of the node with the highest priority
  - Copy the job
  - Shift right to left from this position till the end
  - Decrease the size
  - Return the job

© Confidential Information of PES University. All Rights Reserved - 2026

Priority Queue: ordered array implementation

- Initialize : make the number of elements in the array 0
- Deinitialize : same as initialize!
- Is_empty : check if the number of elements is 0
- Is_full : check if the number of elements equal to the size of the array

© Confidential Information of PES University. All Rights Reserved - 2026

Priority Queue: ordered array implementation

- Enque :
  - Check if full
  - Traverse right to left; shift; find the position to insert
  - insert
  - Increase the size
  - Number of operations proportional to the number of elements
- Deque :
  - Check if empty
  - Find the rightmost element
  - Decrement the size
  - Return the job of the rightmost element before changing size

© Confidential Information of PES University. All Rights Reserved - 2026

Priority Queue: Applications of priority queue :

- CPU Scheduling of processes
- Hospital Emergency Services
- Printer Scheduling
- Network Packet Scheduling
- Dijkstra's Shortest Path Algorithm
- Prim's Minimum Spanning Tree Algorithm
- Event-Driven Simulation
- Huffman Coding

© Confidential Information of PES University. All Rights Reserved - 2026

THANK YOU

N S Kumar
Professor

Department of Computer Science & Engineering (AI & ML)

Introduction

Deque

Data Structures and its Applications (UE25CS52A)

N S Kumar
Professor
Department of Computer Science & Engineering (AI & ML)

© Confidential Information of PES University. All Rights Reserved - 2026

Definition and Properties

- A **deque**
  - Is a data structure
  - Supports add and delete
    - At both the ends
- Pronounced as deck

© Confidential Information of PES University. All Rights Reserved - 2026

Implementation of stack using deque

- Achieved by composition
- Also called embedding|layering|containment
- Is a reuse mechanism
- Avoids reinventing the wheel

© Confidential Information of PES University. All Rights Reserved - 2026

Implementation of stack using deque

- **Deque interface**:
  - void init(dll_t* ptr_dlist);
  - void disp_forward(dll_t* ptr_dlist);
  - void disp_backward(dll_t* ptr_dlist);
  - void add_in_begin(dll_t* ptr_dlist, int key);
  - void deinit(dll_t* ptr_dlist);
  - void add_at_end(dll_t* ptr_dlist, int key);
  - int remove_in_begin(dll_t* ptr_dlist);
  - int remove_at_end(dll_t* ptr_dlist);
  - int is_empty(dll_t *ptr_dlist);
  - int is_full(dll_t *ptr_dlist);

© Confidential Information of PES University. All Rights Reserved - 2026

Implementation of stack using deque

- **Stack interface**:
  - void push(stack_t* ptr_stack, int key);
  - int pop(stack_t* ptr_stack);
  - int is_empty_stack(stack_t* ptr_stack);
  - int is_full_stack(stack_t* ptr_stack);
  - void init_stack(stack_t* ptr_stack);
  - void deinit_stack(stack_t* ptr_stack);

© Confidential Information of PES University. All Rights Reserved - 2026

Implementation of stack using deque

- **Deque interface mapping for stack interface**
  - void push(stack_t* ptr_stack, int key);
    => void add_in_begin(dll_t* ptr_dlist, int key);
  - int pop(stack_t* ptr_stack);
    => void add_in_begin(dll_t* ptr_dlist, int key);
  - int is_empty_stack(stack_t* ptr_stack);
    => int is_empty(dll_t *ptr_dlist);
  - int is_full_stack(stack_t* ptr_stack);
    => int is_full(dll_t *ptr_dlist);
  - void init_stack(stack_t* ptr_stack);
    => void init(dll_t* ptr_dlist);
  - void deinit_stack(stack_t* ptr_stack);
    => void deinit(dll_t* ptr_dlist);

© Confidential Information of PES University. All Rights Reserved - 2026

Stack functions delegate to the deque functions

```
void push(stack_t* ptr_stack, int key)
{
        add_in_begin(&ptr_stack->list_, key);
}
int pop(stack_t* ptr_stack)
{
        return remove_in_begin(&ptr_stack->list_);
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

THANK YOU

N S Kumar
Professor

Department of Computer Science & Engineering (AI & ML)

Introduction

Tree

Data Structures and its Applications (UE25CS52A)

N S Kumar
Professor
Department of Computer Science & Engineering (AI & ML)

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree

1. The number of nodes in a tree having n nodes

a) Depends whether the tree is a binary tree or not

b) Is always n - 1

c) Is 2n as each node has 2 pointers

d) Can vary from 1 to n

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree

1. The number of nodes in a tree having n nodes

a) Depends whether the tree is a binary tree or not

b) Is always n - 1

c) Is 2n as each node has 2 pointers

d) Can vary from 1 to n

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree

2. The longest path from the root to the leaf in any binary tree of n nodes has the length

a) n-1

b) log(n, 2)

c) n

d) n/2

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree

2. The longest path from the root to the leaf in any binary tree of n nodes has the length

a) n-1

b) log(n, 2)

c) n

d) n/2

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree

3. The max number of leaves in a binary tree of n nodes is

a) n-1

b) log(n, 2)

c) Floor(n/2) + 1

d) Floor((n +1)/2)

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree

3. The max number of leaves in a binary tree of n nodes is

a) n-1

b) log(n, 2)

c) Floor(n/2) + 1

d) Floor((n +1)/2)

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree

4. In a binary search tree (key less than the root on the left), the node with the least key

a) Will be a leaf

b) Will not have a left sub tree

c) Will not have a right sub tree

d) Can have both left and right sub trees

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree

4. In a binary search tree (key less than the root on the left), the node with the least key

a) Will be a leaf

b) Will not have a left sub tree

c) Will not have a right sub tree

d) Can have both left and right sub trees

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree

5. The number of comparisons while searching a binary search tree of n nodes in the worst case will be

a) n

b) n/2

c) Log(n, 2)

d) 1

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree

5. The number of comparisons while searching a binary search tree of n nodes in the worst case will be

a) n

b) n/2

c) Log(n, 2)

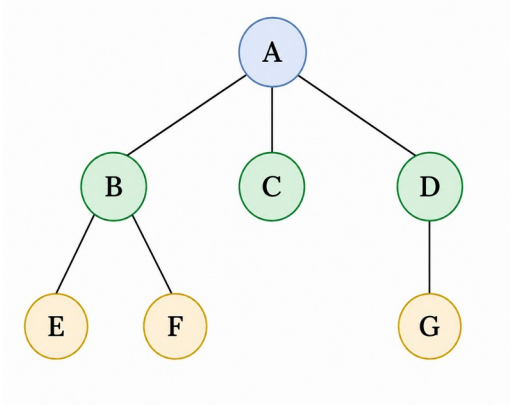
d) 1

© Confidential Information of PES University. All Rights Reserved - 2026

General Tree

Definition and Properties

- A **tree**
  - is a non-linear data structure used to represent hierarchical relationships
  - organizes elements in a parent–child structure.

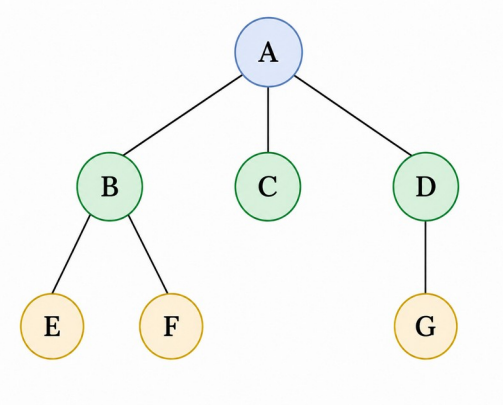


© Confidential Information of PES University. All Rights Reserved - 2026

General Tree

Terminology from genealogy:

- Parent
- Child
- Siblings
- Ancestor
- Descendant
- Uncle

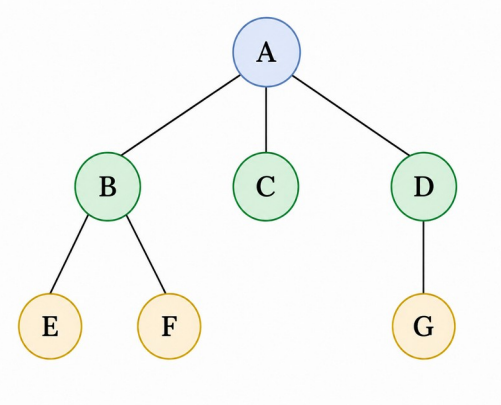


© Confidential Information of PES University. All Rights Reserved - 2026

General Tree

Terminology from botany:

- Root
- Nodes
- Leaves
- Internal nodes
- External nodes
- Branch

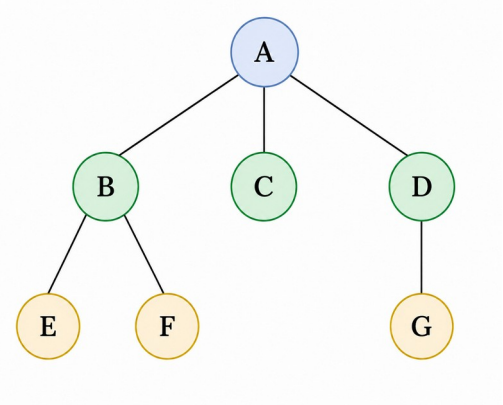


© Confidential Information of PES University. All Rights Reserved - 2026

General Tree

Definition

- A Tree
  - Is either empty
  - Has a root followed by number of trees
    - Called subtrees
- Some more terms
  - Path
  - Level
  - height

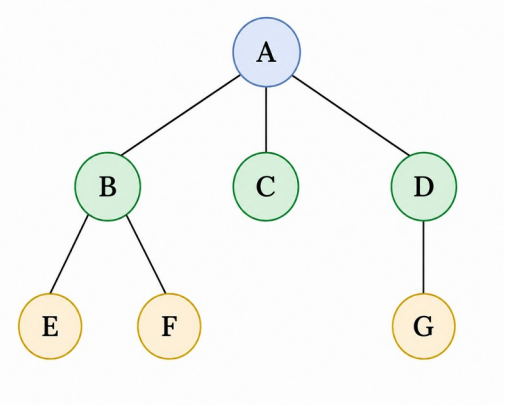


© Confidential Information of PES University. All Rights Reserved - 2026

General Tree

Important properties of trees:

- Acyclic : Can not start from a node, traverse and come back to the same node
- Connected. Any two nodes are connected by a path which may go through the root
- A tree of n nodes has n – 1 edges – neither more not less.
- Add an edge, we will have a cycle. Remove an edge, we will have more than one tree.

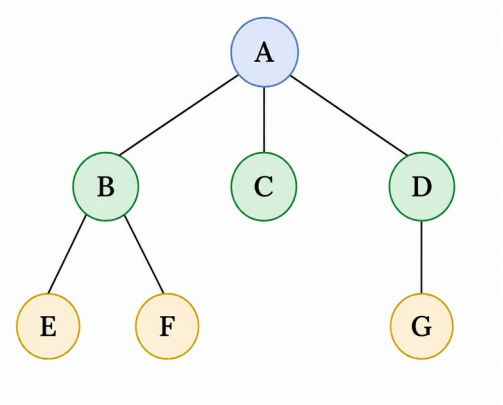


© Confidential Information of PES University. All Rights Reserved - 2026

General Tree

A possible ADT for a general tree :

- Create a tree
- Destroy the tree
- Check empty
- Check full
- Add a node
- Remove a node
- Traverse a tree
- Find root
- Find children
- Check whether leaf
- Find height of the tree
- Find the path from the root to a given node



© Confidential Information of PES University. All Rights Reserved - 2026

General Tree

Applications of the tree

- File System Structure:
- Search Trees:
- Databases, indexing and B-trees:
- Expression Evaluation:
- Compiler design:
- Document representation
- Heap Tree and priority queue:
- Process structure:
- Organization structure
- Decision Trees in AI
- ...

© Confidential Information of PES University. All Rights Reserved - 2026

General Tree

Applications of the tree

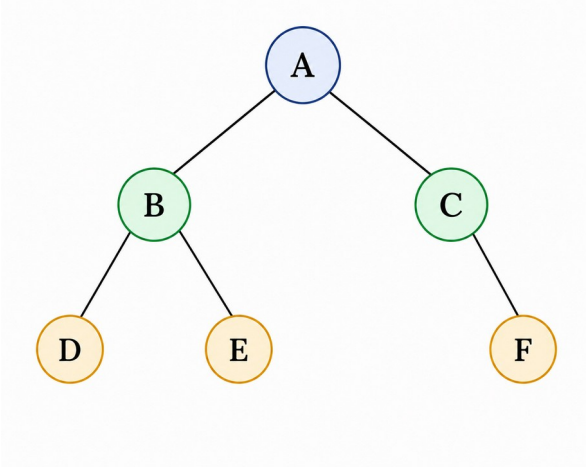
- File System Structure:
- Search Trees:
- Databases, indexing and B-trees:
- Expression Evaluation:
- Compiler design:
- Document representation
- Heap Tree and priority queue:
- Process structure:
- Organization structure
- Decision Trees in AI
- ...

© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree

Definitiion

- A **binary tree** is a tree in which every node has **at most two children**.
- The two children are distinguished as the **left child** and **right child**.
- A binary tree is either:
  - An **empty tree**, or
  - A root node together with two disjoint binary trees called its:
    - **left subtree**
    - **right subtree**



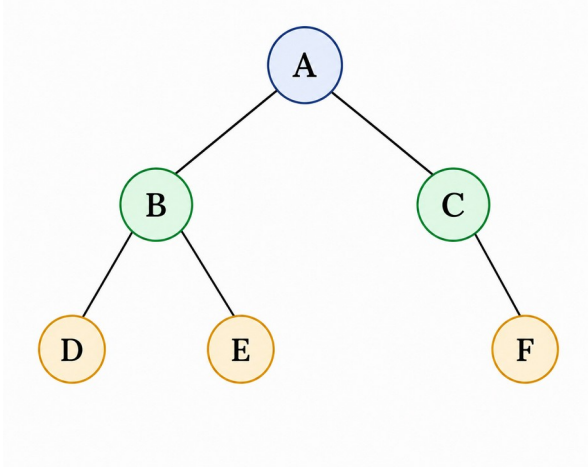
© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree

Inorder traversal of the binary tree

- If tree is not empty
  - Visit the left subtree
  - Visit the root
  - Visit the right subtree

Inorder traversal of this tree:
D B E A C F



© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree

Binary representation of any general tree

- Root of general tree becomes root of the binary tree
- Child of a node in general tree appears to the left of the node in the binary tree
- Sibling of a node in general tree will appear to the right of the node in the binary tree

© Confidential Information of PES University. All Rights Reserved - 2026

Binary Search Tree

- Every node has a key
- Nodes are inserted based on the key
- If the key of a node is less than that of the root, it will be inserted in the left subtree
- Otherwise, it will be inserted in the right subtree

© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree

Binary Search Tree : simple and small interface

- Create a tree
- Destroy the tree
- Insert the key
- Display the tree in inorder

- Find the number of nodes
- Find the number of leaves

© Confidential Information of PES University. All Rights Reserved - 2026

Binary Search Tree : structure of the functions

- Recursive on nodes and not on tree
- Model for recursive routine :
  - Empty tree
  - Tree with left and right subtrees

© Confidential Information of PES University. All Rights Reserved - 2026

THANK YOU

N S Kumar
Professor

Department of Computer Science & Engineering (AI & ML)

Experiential Learning : unit 2

Round Robin Scheduling

N S Kumar
Professor
Department of Computer Science &
Engineering (AI & ML)

© Confidential Information of PES University. All Rights Reserved - 2026

Experiential Learning

Round Robin Scheduling

- Task 1 : Create a circular doubly linked list
  - Input : Doubly linked list – both the header and the implementation
  - Output : Make this doubly linked list Circular
  - ToDo :
    - Support adding a node before the current position
    - Removing a node at the current position
    - Modify display routines suitably
    - Develop a menu driven client code

© Confidential Information of PES University. All Rights Reserved - 2026

Experiential Learning

Round Robin Scheduling

- Task 2 : change the key field from int to a job
  - Input : circular doubly linked list : header and implementation
  - ToDo :
    - Create a type called job
    - Provide interfaces for setting and displaying a job
    - Change the key field in the node structure to job
    - Modify the client code suitably

© Confidential Information of PES University. All Rights Reserved - 2026

Experiential Learning

Round Robin Scheduling

- Task 3 : Implement scheduling
  - Input : circular doubly linked list of jobs
  - ToDo :
    - Each job shall have time for completion and time allotted so far
    - In the client, support adding a job and processing a job
    - On processing a job, update the time allotted for the job by the time slice
    - If the time allotted exceeds the time for completion, remove the job from the data structure

© Confidential Information of PES University. All Rights Reserved - 2026

THANK YOU

Professor

Department of Computer Science & Engineering (AI & ML)